UDC-WQIS Azure Migration Plan

Prepared by: Oli T. Oli / DAPS Analytics **Date:** April 17, 2026 **Current Production:** https://udc-one.vercel.app (Vercel Hobby) **Target:** Microsoft Azure (Gov-ready infrastructure)

---

1. Why Migrate to Azure?

Driver	Detail
Government alignment	Azure holds FedRAMP High + DC OCTO contracts; required for EPA/USGS partnerships
Vercel limitations	Hobby plan: 1 cron/day, 60s function timeout, no SLA, no custom SSL wildcard
Data sovereignty	Azure US East (Virginia) keeps data in-region for DC government compliance
Scalability	White-label multi-watershed deployment needs container orchestration
Grant readiness	EPA EJCPS ($500K) and NSF Smart Communities ($1.5M) grants favor Azure/GovCloud hosting
Cost predictability	Azure for Startups gives $5K–$150K in free credits

---

2. Current Architecture (Vercel)

```
GitHub (main) ──git push──▶ Vercel (auto-deploy)
                              ├── Next.js 16 App (SSR + API routes)
                              ├── Vercel Cron (1x daily USGS ingest)
                              └── Env vars (DATABASE_URL, ANTHROPIC_API_KEY, etc.)

Neon PostgreSQL (serverless) ◀── DATABASE_URL connection
Google Cloud Run ◀── WQIS ADK Agent (Python)
```

---

3. Target Architecture (Azure)

```
GitHub (main) ──push──▶ GitHub Actions CI ──deploy──▶ Azure Container Apps
                                                       ├── Next.js 16 container (port 3000)
                                                       ├── Custom domain + managed SSL
                                                       └── Env vars from Azure Key Vault

Azure Database for PostgreSQL (Flexible Server) ◀── DATABASE_URL
Azure Functions (Timer Triggers) ──POST──▶ /api/ingest?source=usgs|epa|wqp
Azure Blob Storage ─── CSV/JSON uploads + data exports
Azure AI Services ─── (optional) replace Anthropic with Azure OpenAI
Azure Monitor + App Insights ─── logging, alerts, uptime
```

---

4. Migration Phases

Phase 1: Database Migration (Week 1)

Neon PostgreSQL → Azure Database for PostgreSQL Flexible Server

Step	Action	Notes
1.1	Create Azure PostgreSQL Flexible Server (Burstable B1ms, 1 vCore, 2GB)	~$13/mo

1.2	Export Neon data: `pg_dump --no-owner --no-acl -Fc udc_water > udc_backup.dump`	Run from local machine with Neon connection string
1.3	Import to Azure: `pg_restore -h <azure-host> -U <admin> -d udc_water udc_backup.dump`	
1.4	Verify row counts: `stations` (12), `readings` (10K+), `measurements` (12K+), `parameters` (25), `ingestion_log`	
1.5	Update `DATABASE_URL` env var on Vercel temporarily to test Azure DB with current frontend	Zero-downtime validation
1.6	Enable Azure firewall rules for Vercel IPs (temporary) and future Container Apps subnet	
1.7	Enable automated backups (7-day retention) + geo-redundancy	

Database connection string format:

```
postgresql://<user>:<password>@<server>.postgres.database.azure.com:5432/udc_water?sslmode=require
```

Code change required: Remove `@neondatabase/serverless` driver, switch to standard `pg` (node-postgres) since Azure PostgreSQL uses standard connections, not WebSocket.

```
- import { neon } from '@neondatabase/serverless';
+ import { Pool } from 'pg';

- const sql = neon(process.env.DATABASE_URL!);
+ const pool = new Pool({ connectionString: process.env.DATABASE_URL });
+ const sql = (query: string, params?: any[]) => pool.query(query, params).then(r => r.rows);
```

File affected: `src/lib/db.ts` (dual-driver logic already exists — add `pg` as third driver)

Phase 2: Container Apps Deployment (Week 2)

Vercel → Azure Container Apps

Step	Action	Notes
2.1	Create Azure Container Registry (ACR) — Basic tier	~$5/mo
2.2	Build Docker image: `docker build -t udc-wqis:latest .`	Dockerfile already exists
2.3	Push to ACR: `az acr build --registry udcwqis --image udc-wqis:latest .`	
2.4	Create Container Apps Environment (Consumption plan)	Pay-per-use, scales to zero
2.5	Deploy container app with env vars from Key Vault	
2.6	Configure custom domain + managed TLS certificate	
2.7	Set up health probe on `/api/health`	Already implemented
2.8	Configure ingress (external, port 3000, HTTPS only)	

Azure CLI deployment:

```
# Create resource group
az group create --name rg-udc-wqis --location eastus

# Create Container Apps environment
az containerapp env create \
  --name env-udc-wqis \
  --resource-group rg-udc-wqis \
  --location eastus

# Deploy container
az containerapp create \
  --name udc-wqis \
  --resource-group rg-udc-wqis \
  --environment env-udc-wqis \
  --image udcwqis.azurecr.io/udc-wqis:latest \
  --target-port 3000 \
  --ingress external \
  --min-replicas 0 \
  --max-replicas 3 \
  --cpu 0.5 --memory 1Gi \
  --env-vars \
    DATABASE_URL=secretref:database-url \
    ANTHROPIC_API_KEY=secretref:anthropic-key \
    ADMIN_API_KEY=secretref:admin-key \
    INGEST_API_KEY=secretref:ingest-key
```

Dockerfile update needed:

```
# Add to existing Dockerfile
ENV PORT=3000
ENV HOSTNAME="0.0.0.0"
EXPOSE 3000
```

Phase 3: Cron Jobs Migration (Week 2)

Vercel Cron → Azure Functions Timer Triggers

Step	Action	Notes
3.1	Create Azure Functions App (Node.js 20, Consumption plan)	Free tier: 1M executions/mo
3.2	Create 3 timer-trigger functions	See schedule below
3.3	Each function calls `POST https://<app-url>/api/ingest?source=<source>` with `INGEST_API_KEY`	
3.4	Remove `crons` from `vercel.json` (no longer needed)	

Cron schedule (matching current):

Function	Schedule (NCRONTAB)	Current Vercel Cron
`usgs-ingest`	`0 0 6 * * *`	`0 6 * * *` (daily 06:00 UTC)
`epa-ingest`	`0 0 7 * * *`	Manual only (was daily, disabled)
`wqp-ingest`	`0 0 8 * * 1`	Manual only (was weekly Mon, disabled)

Sample Azure Function (TypeScript):

```
// usgs-ingest/index.ts
import { app } from '@azure/functions';

app.timer('usgs-ingest', {
  schedule: '0 0 6 * * *',
  handler: async (myTimer, context) => {
    const res = await fetch(`${process.env.APP_URL}/api/ingest?source=usgs`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${process.env.INGEST_API_KEY}` }
    });
    const data = await res.json();
    context.log(`USGS ingest: ${data.status}, records: ${data.count}`);
  }
});
```

Advantage over Vercel: No 1-cron limit, 10-minute timeout (vs 60s), retry policies, dead-letter queues.

Phase 4: Secrets & Monitoring (Week 3)

4.1 Azure Key Vault

Secret	Source
`database-url`	Azure PostgreSQL connection string
`anthropic-key`	Anthropic API key (existing)
`admin-key`	Admin panel access key (existing)
`ingest-key`	Ingestion API key (existing)
`wqis-agent-url`	Cloud Run WQIS agent URL (keep or migrate later)

4.2 Azure Monitor + Application Insights

Monitor	Config
Uptime check	Ping `/api/health` every 5 min
Alert: DB connection failure	`/api/health` returns `degraded`
Alert: Ingestion failure	`ingestion_log` status = `error` for >24h
Alert: High response time	P95 > 3s on any API route
Log Analytics	Stream container logs + function logs
Dashboard	Station count, ingestion success rate, API latency

Phase 5: CI/CD Pipeline Update (Week 3)

Update GitHub Actions to deploy to Azure Container Apps:

```
# .github/workflows/deploy-azure.yml
name: Build & Deploy to Azure
on:
```

```yaml
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Azure
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Build and push to ACR
        uses: azure/docker-login@v2
        with:
          login-server: udcwqis.azurecr.io
          username: ${{ secrets.ACR_USERNAME }}
          password: ${{ secrets.ACR_PASSWORD }}
      - run: |
          docker build -t udcwqis.azurecr.io/udc-wqis:${{ github.sha }} .
          docker push udcwqis.azurecr.io/udc-wqis:${{ github.sha }}

      - name: Deploy to Container Apps
        uses: azure/container-apps-deploy-action@v2
        with:
          acrName: udcwqis
          containerAppName: udc-wqis
          resourceGroup: rg-udc-wqis
          imageToDeploy: udcwqis.azurecr.io/udc-wqis:${{ github.sha }}
```

Phase 6: DNS Cutover & Vercel Decommission (Week 4)

Step	Action
6.1	Verify Azure deployment fully functional (all 14 API endpoints, cron running, AI chat working)
6.2	Run parallel: keep Vercel live + Azure live for 72h, compare `/api/health` responses
6.3	Point `udc-one.vercel.app` custom domain (if any) to Azure Container Apps
6.4	Update `WQIS_AGENT_URL` if migrating Cloud Run agent to Azure
6.5	Delete Vercel project after 7-day soak period
6.6	Delete Neon PostgreSQL database after confirming Azure DB has all data

---

5. Cost Estimate (Monthly)

Service	Tier	Est. Cost
Azure Container Apps	Consumption (0.5 vCPU, 1GB)	$5–15
Azure PostgreSQL Flexible	Burstable B1ms (1 vCore, 2GB)	$13

Azure Container Registry	Basic	$5
Azure Functions	Consumption (3 timers)	~$0 (free tier)
Azure Key Vault	Standard	~$0.03/secret/mo
Azure Monitor	Basic	Free tier
Azure Blob Storage	Hot (if used for exports)	~$1
Total		**~$25–35/mo**

vs. current Vercel Hobby: $0/mo (free) but with severe limitations (1 cron, 60s timeout, no SLA)

Azure for Startups credit: Apply for $5K–$150K free credits via Microsoft for Startups Founders Hub (Oli qualifies as founder).

---

6. Code Changes Summary

File	Change	Effort
`src/lib/db.ts`	Add `pg` driver alongside Neon + SQLite	Small
`package.json`	Add `pg` + `@types/pg`, optionally remove `@neondatabase/serverless` + `ws`	Small
`Dockerfile`	Add `HOSTNAME=0.0.0.0`, verify `standalone` output	Minimal
`vercel.json`	Remove `crons` section (optional, keep for fallback)	Minimal
`.github/workflows/deploy-azure.yml`	New CI/CD workflow	Medium
`next.config.ts`	Verify `output: 'standalone'` is set	Check only

Total code changes: ~50 lines modified, 1 new file (Azure CI workflow).

---

7. Risk Mitigation

Risk	Mitigation
Data loss during DB migration	`pg_dump` + verify row counts before cutover
Ingestion downtime	Run Vercel cron + Azure Functions in parallel for 1 week
Cold start latency	Set `min-replicas: 1` during business hours (adds ~$10/mo)
Neon WebSocket driver incompatibility	Standard `pg` driver works with Azure PostgreSQL out of the box
Docker image size	Current Dockerfile uses multi-stage build, already optimized
SQLite in container	Local dev continues with SQLite; production uses Azure PostgreSQL

---

8. Future Azure Expansions

Capability	Azure Service	When
WQIS AI Agent	Azure Container Apps (migrate from Cloud Run)	After MVP stable
Vector search (RAG)	Azure AI Search or pgvector on Azure PostgreSQL	Phase 10
User auth	Azure AD B2C or NextAuth with Azure provider	When needed

Multi-watershed white-label	Multiple Container Apps per customer + shared DB	Business phase
Real-time alerts	Azure Event Grid + Logic Apps → email/SMS	After monitoring stable
Blob storage for uploads	Azure Blob Storage (replace in-memory CSV handling)	Phase 2+

---

9. Pre-Migration Checklist

- [ ] Apply for Azure for Startups credits
- [ ] Create Azure account + subscription
- [ ] Create resource group `rg-udc-wqis` in East US
- [ ] Export Neon database backup
- [ ] Test Docker build locally: `docker build -t udc-wqis . && docker run -p 3000:3000 udc-wqis`
- [ ] Add `pg` package: `npm install pg @types/pg`
- [ ] Update `src/lib/db.ts` with Azure PostgreSQL driver
- [ ] Create Azure PostgreSQL instance + import data
- [ ] Test locally with Azure DB connection string
- [ ] Create ACR + push image
- [ ] Deploy to Container Apps
- [ ] Verify all 14 API endpoints
- [ ] Set up Azure Functions for cron
- [ ] Verify USGS ingestion runs successfully
- [ ] Set up monitoring + alerts
- [ ] Run parallel with Vercel for 72h
- [ ] DNS cutover
- [ ] Decommission Vercel + Neon

This plan is designed for a solo developer (Oli) to execute in 3–4 weeks with minimal code changes and zero data loss.